

Sanyar's TaskWriteup

This task is a Django backend for managing Cluster, Namespace and App resources on Kubernetes, plus a Backup system using Celery for async and periodic tasks. Both parts share Redis as cache and Celery broker.

Cluster POST only saves to database, no k3s connection is made, and token is never returned in any response since it is sensitive data.

Namespace POST first checks if the cluster exists, then creates the namespace in k3s, and only after success saves it to database. Duplicate namespace (checked in DB and via k3s 409) returns 409 Conflict, with proper codes also for cluster not found (404), permission (403) and k3s unreachable (502). Namespace GET follows Database as Source of Truth, so it never shows namespaces that exist only in k3s but were not created by backend.

For DELETE namespace I handle two edge cases:

- Concurrent delete requests: solved with `select_for_update()` inside a transaction plus a status field (active/deleting), so a second parallel request gets 409 instead of deleting twice.
- Crash between k3s delete and DB delete: solved by setting status to deleting before calling k3s, so if backend crashes mid-way the record stays in a clearly stuck state instead of looking active. A periodic Celery Beat task `reconcile_namespaces` runs every 5 minutes to clean up records that no longer exist in k3s.

App has full CRUD, and pod status is always read live from Kubernetes as required, cached in Redis for 60 seconds to reduce load on the k3s API. On App UPDATE, if the k3s call fails, in-memory changes are rolled back so DB stays consistent.

For throttling I added a simple Redis-based counter per IP limiting namespace creation to 5 requests per 60 seconds, returning 429 when exceeded — not a full rate-limit library but enough to show the concept.

For Backup, POST `/backup/` creates a record with status pending and returns `backup_id` immediately, while the real backup (finding the pod, copying the file, saving under `/backups/{app_id}/{date}/{backup_id}.tar.gz`) runs asynchronously in the Celery worker task `backup_task`, so the HTTP request never waits. Status can be checked with GET `/backup/details/{backup_id}/`, and all backups for an app listed with GET `/backup/list/?app_id=`.

A periodic task `check_stale_backups` runs hourly and marks any backup stuck in pending for more than 24 hours as failed with an error message, and `backup_task` itself uses `autoretry_for` with backoff and max 3 retries for transient errors.

Periodic backups are created by sending a `schedule` field (cron expression) in POST `/backup/`, using `django-celery-beat` to dynamically create a `PeriodicTask` and `CrontabSchedule`. Each scheduled run calls `run_scheduled_backup`, which creates a fully independent new Backup

record with its own `backup_id`, so it shows correctly in both the status and list endpoints, same as an instant backup.

As a bonus, live app status is cached in Redis with a 60 second TTL through `get_cached_app_status()`, used only for app status data and kept isolated from the actual backup records and files.